

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №4
дисциплины
«Искусственный интеллект и машинное обучение»

Выполнил:
Червиченко Иван Денисович
2 курс, группа ИВТ-б-о-24-2,
09.03.01 «Информатика и
вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная
форма обучения

(подпись)

Проверил:
Доцент департамента цифровых,
робототехнических систем и
электроники института
перспективной инженерии Воронкин
Р.А

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2026 г.

Цель работы: познакомить с основами работы с библиотекой pandas, в частности, со структурой данных Series.

Ссылка на репозиторий: <https://github.com/progwar/ailab4>

Ход работы:

1. Был создан общедоступный репозиторий, в котором использована лицензия MIT и язык программирования Python.

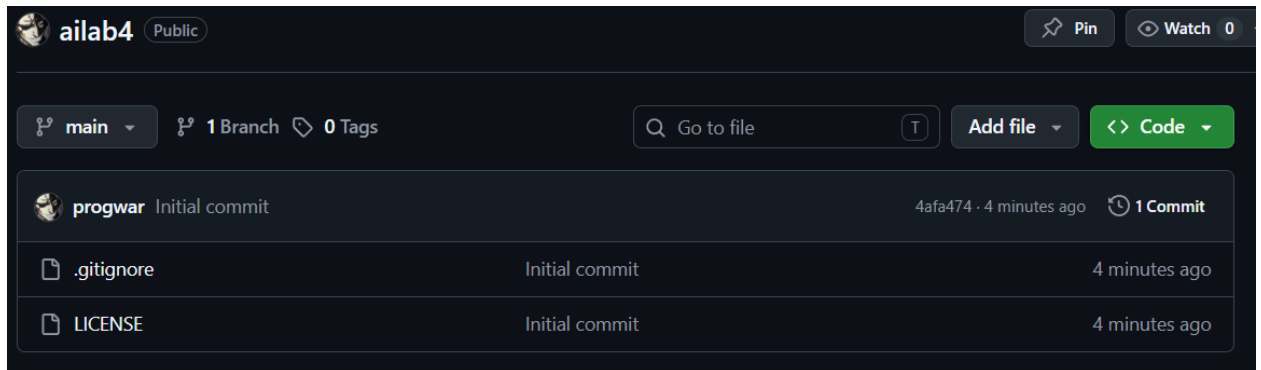


Рисунок 1. Созданный репозиторий

2. Репозиторий был клонирован на рабочий компьютер.

```
PS D:\github> git clone "https://github.com/progwar/ailab4.git"
Cloning into 'ailab4'...
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (4/4), done.
```

Рисунок 2. Клонирование репозитория

2. Файл .gitignore был дополнен необходимыми правилами.

```
# Sphinx documentation
docs/_build/

# PyBuilder
.pybuilder/
target/

# Jupyter Notebook
.ipynb_checkpoints/
*.ipynb_checkpoints*
*/.ipynb_checkpoints/*
.ipynb_checkpoints/*
*.ipynb.json
*/.ipynb_checkpoints/
~*.ipynb
*.ipynb.py
.ipynb_checkpoints/
*.pkl
*.pickle

# JupyterLab
.jupyter/
jupyter_config.json
jupyter_notebook_config.json
jupyter_notebook_config.py
jupyterlab-workspace.json

# IPython
profile_default/
ipython_config.py

# pyenv
# For a library or package, you might want to ignore these files since the code is
# intended to run in multiple environments; otherwise, check them in:
# .python-version

# pipenv
# According to pya/pipenv#598, it is recommended to include Pipfile.lock in version control.
```

Рисунок 3. Дополненный .gitignore файл

4. Были проработаны примеры лабораторной работы.

```
[2]: import pandas as pd
import numpy as np

[3]: s1 = pd.Series([1, 2, 3, 4, 5])
print(s1)

0    1
1    2
2    3
3    4
4    5
dtype: int64
```

Рисунок 4. Пример 1

```
s2 = pd.Series([1, 2, 3, 4, 5], ['a', 'b', 'c', 'd', 'e'])
print(s2)

a    1
b    2
c    3
d    4
e    5
dtype: int64
```

Рисунок 5. Пример 2

```
: ndarr = np.array([1, 2, 3, 4, 5])
type(ndarr)

: numpy.ndarray
```

Рисунок 6. Пример 3

```
s3 = pd.Series(ndarr, ['a', 'b', 'c', 'd', 'e'])
print(s3)
```

a	1
b	2
c	3
d	4
e	5

dtype: int64

Рисунок 7. Пример 4

```
d = {'a':1, 'b':2, 'c':3}
s4 = pd.Series(d)
print(s4)
```

a	1
b	2
c	3

dtype: int64

Рисунок 8. Пример 5

```
a = 7
s5 = pd.Series(a, ['a', 'b', 'c'])
print(s5)
```

a	7
b	7
c	7

dtype: int64

Рисунок 9. Пример 6

```
s6 = pd.Series([1, 2, 3, 4, 5], ['a', 'b', 'c', 'd', 'e'])
print(s6['c'])
print(s6.iloc[2])
```

3
3

Рисунок 10. Пример 7

```
print(s6['d'])
```

4

Рисунок 11. Пример 8

```
print(s6[:2])
```

a	1
b	2

dtype: int64

Рисунок 12. Пример 9

```
s6[s6<=3]
```

a	1
b	2
c	3

dtype: int64

Рисунок 13. Пример 10

```

: s7 = pd.Series([10, 20, 30, 40, 50], ['a', 'b', 'c', 'd', 'e'])
  print(s6 + s7)
  print(s6*3)
a    11
b    22
c    33
d    44
e    55
dtype: int64
a     3
b     6
c     9
d    12
e    15
dtype: int64

```

Рисунок 14. Пример 11

```

s = pd.Series([10, 20, 30, 40, 50], index=['a', 'b', 'c', 'd', 'e'])
print(s.iloc[0])
print(s.iloc[2])
print(s.iloc[-1])
10
30
50

```

Рисунок 15. Пример 12

```

print(s.iloc[1:3])
print(s.loc["b":"d"])
b    20
c    30
dtype: int64
b    20
c    30
d    40
dtype: int64

```

Рисунок 16. Пример 13

```

s = pd.Series([10,25,8,30,15], index=['a','b','c','d','e'])
filtered_s=s[s>10]
print(filtered_s)
b    25
d    30
e    15
dtype: int64

```

Рисунок 17. Пример 14

```

: print(s>10)
a    False
b     True
c    False
d     True
e     True
dtype: bool

```

Рисунок 18. Пример 15

```

filtered_s=s[(s>=10)&(s<=30)]
print(filtered_s)

a    10
b    25
d    30
e    15
dtype: int64

```

Рисунок 19. Пример 16

```

filtered_s = s[s.index.isin(['b','d'])]
print(filtered_s)

b    25
d    30
dtype: int64

```

Рисунок 20. Пример 17

```

s_with_nan=pd.Series([10,None,8,30,None],index=['a','b','c','d','e'])
filtered_s=s_with_nan[s_with_nan.notnull()]
print(filtered_s)

a    10.0
c     8.0
d    30.0
dtype: float64

```

Рисунок 21. Пример 18

```

: s=pd.Series([10,20,30,40],index=['a','b','c','d'])
  s.loc['b']=25
  print(s)

a    10
b    25
c    30
d    40
dtype: int64

```

Рисунок 22. Пример 19

```

s=pd.Series([10,20,30,40],index=['a','b','c','d'])
s[s>30]+=10
print(s)

a    10
b    20
c    30
d    50
dtype: int64

```

Рисунок 23. Пример 20

```

: s.loc[['a','c']]=[100,200]
  print(s)

a    100
b     20
c    200
d     50
dtype: int64

```

Рисунок 24. Пример 21

```
s = s.apply(lambda x:x*2)
print(s)

a    200
b     40
c    400
d    100
dtype: int64
```

Рисунок 25. Пример 22

```
s_with_nan=pd.Series([10,None,8,30,None],index=['a','b','c','d','e'])
s_filled = s_with_nan.fillna(0)
print(s_filled)

a    10.0
b     0.0
c     8.0
d    30.0
e     0.0
dtype: float64
```

Рисунок 26. Пример 23

```
s=pd.Series([10,20,30,40,50,60,70],index=['a','b','c','d','e','f','g'])
print(s.head(3))

a    10
b    20
c    30
dtype: int64
```

Рисунок 27. Пример 24

```
print(s.head())

a    10
b    20
c    30
d    40
e    50
dtype: int64
```

Рисунок 29. Пример 25

```
print(s.tail(3))

e    50
f    60
g    70
dtype: int64
```

Рисунок 30. Пример 26

```
print(s.tail())

c    30
d    40
e    50
f    60
g    70
dtype: int64
```

Рисунок 31. Пример 27

```
s=pd.Series([10,20,30,40,50],index=['a','b','c','d','e'])
print(s.index)
```

Index(['a', 'b', 'c', 'd', 'e'], dtype='str')

Рисунок 32. Пример 28

```
print(s.index[0])
print(s.index[-1])
```

a
e

Рисунок 33. Пример 29

```
print(s.values)
print(s.values[0])
print(s.values.mean())
```

[10 20 30 40 50]
10
30.0

Рисунок 34. Пример 30

```
s=pd.Series([10,20,30,40,50])
print(s.dtype)
```

int64

Рисунок 35. Пример 31

```
s1=pd.Series([1.0,2.5,3.7])
print(s1.dtype)

s2=pd.Series(["apple","bananan","cherry"])
print(s2.dtype)

s3=pd.Series([True,False,True])
print(s3.dtype)
```

float64
str
bool

Рисунок 36. Пример 32

```
s1_int=s1.astype(int)
print(s1_int)
print(s1_int.dtype)
```

0 1
1 2
2 3
dtype: int64
int64

Рисунок 37. Пример 33

```
s4=pd.Series([10,"apple",3.14,True])
print(s4.dtype)
```

object

Рисунок 38. Пример 34

```
s = pd.Series([10,np.nan,30,None,50])
print(s.isnull())
```

```
0    False
1     True
2    False
3     True
4    False
dtype: bool
```

Рисунок 39. Пример 35

```
print(s.notnull())
```

```
0     True
1    False
2     True
3    False
4     True
dtype: bool
```

Рисунок 40. Пример 36

```
filtered_s = s[s.notnull()]
print(filtered_s)
```

```
0    10.0
2    30.0
4    50.0
dtype: float64
```

Рисунок 41. Пример 37

```
mis_s = s[s.isnull()]
print(mis_s)
```

```
1    NaN
3    NaN
dtype: float64
```

Рисунок 42. Пример 38

```
s = pd.Series([10, np.nan, 30, None, 50])
s_filled = s.fillna(0)
print(s_filled)
```

```
0    10.0
1     0.0
2    30.0
3     0.0
4    50.0
dtype: float64
```

Рисунок 43. Пример 39

```
s = pd.Series([10, np.nan, 30, None, 50])
s_filled = s.fillna(s.mean())
print(s_filled)
```

```
0    10.0
1    30.0
2    30.0
3    30.0
4    50.0
dtype: float64
```

Рисунок 44. Пример 40

```
s_clean=s.dropna()
print(s_clean)

0    10.0
2    30.0
4    50.0
dtype: float64
```

Рисунок 45. Пример 41

```
s = pd.Series([10, 20, 30, 40, 50])
s_mult = s*2
print(s_mult)

0    20
1    40
2    60
3    80
4   100
dtype: int64
```

Рисунок 46. Пример 42

```
s1 = pd.Series([1, 2, 3, 4, 5], index = ['a','b','c','d','e'])
s2 = pd.Series([10, 20, 30, 40, 50], index = ['a','b','c','d','e'])
s_sum = s1+s2
print(s_sum)

a    11
b    22
c    33
d    44
e    55
dtype: int64
```

Рисунок 47. Пример 43

```
s3 =pd.Series([100,200,300], index = ['a','b','f'])
s_res = s1+s3
print(s_res)

a    101.0
b    202.0
c      NaN
d      NaN
e      NaN
f      NaN
dtype: float64
```

Рисунок 48. Пример 44

```
s_res = s1.add(s3, fill_value = 0)
print(s_res)

a    101.0
b    202.0
c      3.0
d      4.0
e      5.0
f    300.0
dtype: float64
```

Рисунок 49. Пример 45

```

s = pd.Series([1,2,3,4,5])
s_squar = s.apply(lambda x:x**2)
print(s_squar)

0    1
1    4
2    9
3   16
4   25
dtype: int64

```

Рисунок 50. Пример 46

```

def cust_func(x):
    return f"Value:{x}"
s_trans = s.apply(cust_func)
print(s_trans)

0    Value:1
1    Value:2
2    Value:3
3    Value:4
4    Value:5
dtype: str

```

Рисунок 51. Пример 47

```

s_log=s.apply(np.log)
print(s_log)

0    0.000000
1    0.693147
2    1.098612
3    1.386294
4    1.609438
dtype: float64

```

Рисунок 52. Пример 48

```

s_text = pd.Series(["apple","bananan","cherry"])
s_upp = s_text.apply(str.upper)
print(s_upp)

0    APPLE
1  BANANAN
2   CHERRY
dtype: str

```

Рисунок 53. Пример 49

```

s=pd.Series([10,20,30,40,50])
print(s.sum())
print(s.mean())
print(s.max())
print(s.min())

150
30.0
50
10

```

Рисунок 54. Пример 50

```

s_with_nan = pd.Series([10,20,None,40,50])
print(s_with_nan.sum())
print(s_with_nan.mean())

120.0
30.0

```

Рисунок 55. Пример 51

```
print(s.describe())
```

count	5.000000
mean	30.000000
std	15.811388
min	10.000000
25%	20.000000
50%	30.000000
75%	40.000000
max	50.000000
dtype:	float64

Рисунок 56. Пример 52

```
s_text = pd.Series(["apple", "banana", "apple", "cherry", "banana"])
print(s_text.describe())
```

count	5
unique	3
top	apple
freq	2
dtype:	object

Рисунок 57. Пример 53

```

s = pd.Series([1,2,3,4,5])
s_log = np.log(s)
print(s_log)
print("////////")
s_exp = np.exp(s)
print(s_exp)
print("////////")
s_sqrt = np.sqrt(s)
print(s_sqrt)

```

```

0    0.000000
1    0.693147
2    1.098612
3    1.386294
4    1.609438
dtype: float64
////////
0     2.718282
1     7.389056
2    20.085537
3    54.598150
4   148.413159
dtype: float64
////////
0     1.000000
1     1.414214
2     1.732051
3     2.000000
4     2.236068
dtype: float64

```

```

print(np.sin(s))
print(np.cos(s))
print(np.abs(s))

```

```

0    0.841471
1    0.909297
2    0.141120
3   -0.756802
4   -0.958924
dtype: float64
0    0.540302
1   -0.416147
2   -0.989992
3   -0.653644
4    0.283662
dtype: float64
0     1
1     2
2     3
3     4
4     5
dtype: int64

```

Рисунок 58. Пример 54-55

```

: s_with_nan= pd.Series([1,np.nan,4,9])
  print(np.sqrt(s_with_nan))

0    1.0
1    NaN
2    2.0
3    3.0
dtype: float64

: s = pd.Series([1, 2, 3, 4], index = ['a','b','c','d'])
  s.index = ['x','y','z','w']
  print(s)

x    1
y    2
z    3
w    4
dtype: int64

: s_reset=s.reset_index()
  print(s_reset)

   index  0
0     x   1
1     y   2
2     z   3
3     w   4

: s_uniq = pd.Series([10,20,30], index=['a','b','c'])
  print(s_uniq.index.is_unique)

True

: s_dupl=pd.Series([10,20,30], index=['a','b','a'])
  print(s_dupl.index.is_unique)

False

: print(s_dupl.loc['a'])

a    10
a    30
dtype: int64

: s_reset = s_dupl.reset_index(drop=True)
  print(s_reset)

0    10
1    20
2    30
dtype: int64

: s_dupl.index = [f"{i}_{n}" for n, i in enumerate(s_dupl.index)]
  print(s_dupl)

a_0    10
b_1    20
a_2    30
dtype: int64

```

Рисунок 59. Пример 56-63

```

s = pd.Series([10,20,30,40], index=['d','b','a','c'])
s_sorted = s.sort_index()
print(s_sorted)

a    30
b    20
c    40
d    10
dtype: int64

s = pd.Series([10,20,30,40], index=['d','b','a','c'])
s_sorted_desc = s.sort_index(ascending=False)
print(s_sorted_desc)

d    10
c    40
b    20
a    30
dtype: int64

s_sort_values = s.sort_values()
print(s_sort_values)

d    10
b    20
a    30
c    40
dtype: int64

s_sort_values_des = s.sort_values(ascending=False)
print(s_sort_values_des)

c    40
a    30
b    20
d    10
dtype: int64

s_nan = pd.Series([10,None,30, None,20], index=['a','b','c','d','e'])
print(s_nan.sort_values())

a    10.0
e    20.0
c    30.0
b     NaN
d     NaN
dtype: float64

print(s_nan.sort_values(na_position='first'))

b     NaN
d     NaN
a    10.0
e    20.0
c    30.0
dtype: float64

dates=pd.date_range(start='2024-03-01',periods=5,freq='D')
s=pd.Series([100,105,102,98,110], index=dates)
print(s)

2024-03-01    100
2024-03-02    105
2024-03-03    102
2024-03-04     98
2024-03-05    110
Freq: D, dtype: int64

```

Рисунок 60. Пример 64-70

```
print(s['2024-03-03'])
print(s['2024-03-02':'2024-03-04'])
```

```
102
2024-03-02    105
2024-03-03    102
2024-03-04     98
Freq: D, dtype: int64
```

```
dates = pd.date_range(start='2024-03-01', periods=5, freq='h')
s = pd.Series([10, 15, 12, 8, 20], index=dates)
print(s)
```

```
2024-03-01 00:00:00    10
2024-03-01 01:00:00    15
2024-03-01 02:00:00    12
2024-03-01 03:00:00     8
2024-03-01 04:00:00    20
Freq: h, dtype: int64
```

```
s = pd.Series([10, 15, 12, ], index=['2024-03-01', '2024-03-02', '2024-03-03'])
s.index = pd.to_datetime(s.index)
print(s)
```

```
2024-03-01    10
2024-03-02    15
2024-03-03    12
dtype: int64
```

```
print(s.resample('D').mean())
```

```
2024-03-01    10.0
2024-03-02    15.0
2024-03-03    12.0
Freq: D, dtype: float64
```

```
print(s[s.index.year==2024])
```

```
2024-03-01    10
2024-03-02    15
2024-03-03    12
dtype: int64
```

```
print(s.shift(1))
```

```
2024-03-01     NaN
2024-03-02    10.0
2024-03-03    15.0
dtype: float64
```

```
s = pd.Series([10,20,30,40,50,60,70], index=pd.date_range("2024-03-01", periods=7, freq='D'))
s_ma = s.rolling(window=3).mean()
print(s_ma)
```

```
2024-03-01     NaN
2024-03-02     NaN
2024-03-03    20.0
2024-03-04    30.0
2024-03-05    40.0
2024-03-06    50.0
2024-03-07    60.0
Freq: D, dtype: float64
```

Рисунок 61. Пример 70-77

```
s_ma5 = s.rolling(window=5).mean()
print(s_ma5)
```

```
2024-03-01    NaN
2024-03-02    NaN
2024-03-03    NaN
2024-03-04    NaN
2024-03-05    30.0
2024-03-06    40.0
2024-03-07    50.0
Freq: D, dtype: float64
```

```
s_ma2 = s.rolling(window=3, min_periods=1).mean()
print(s_ma2)
```

```
2024-03-01    10.0
2024-03-02    15.0
2024-03-03    20.0
2024-03-04    30.0
2024-03-05    40.0
2024-03-06    50.0
2024-03-07    60.0
Freq: D, dtype: float64
```

```
import matplotlib.pyplot as plt
plt.figure(figsize=(8,4))
plt.plot(s,label=r'Исходные данные', marker="o")
plt.plot(s.rolling(window=3).mean(),label=r'Скользящее среднее (3)', linestyle='--', color='red')
plt.xlabel(r"Дата")
plt.ylabel(r"Значение")
plt.title(r"Скользящее среднее в Series")
plt.legend()
plt.grid()
plt.show()
```



Рисунок 62. Пример 78-80

```
s=pd.Series([100,110,120,90,150])  
print(s.pct_change())
```

```
0      NaN  
1    0.100000  
2    0.090909  
3   -0.250000  
4    0.666667  
dtype: float64
```

```
print(s.pct_change( periods=2))
```

```
0      NaN  
1      NaN  
2    0.200000  
3   -0.181818  
4    0.250000  
dtype: float64
```

Рисунок 63. Пример 81-82

```

: dates=pd.date_range(start="2024-01-01",periods=30, freq="D")
prices = pd.Series(np.random.randint(90,110, size=30), index=dates)
returns =prices.pct_change()*100
plt.figure(figsize=(10,5))
plt.plot(prices,label=r"Цена акции", marker="o")
plt.ylabel(r"Цена")
plt.xlabel(r"Дата")
plt.title(r"График цены акции")
plt.figure(figsize=(10,5))
plt.bar(returns.index,returns, color="red", alpha=0.7, label=r"")
plt.axhline(0,color='black', linestyle='--')
plt.ylabel(r"Изменение (%)")
plt.xlabel(r"Дата")
plt.title(r"Процентное изменение цены акции")
plt.show()

```

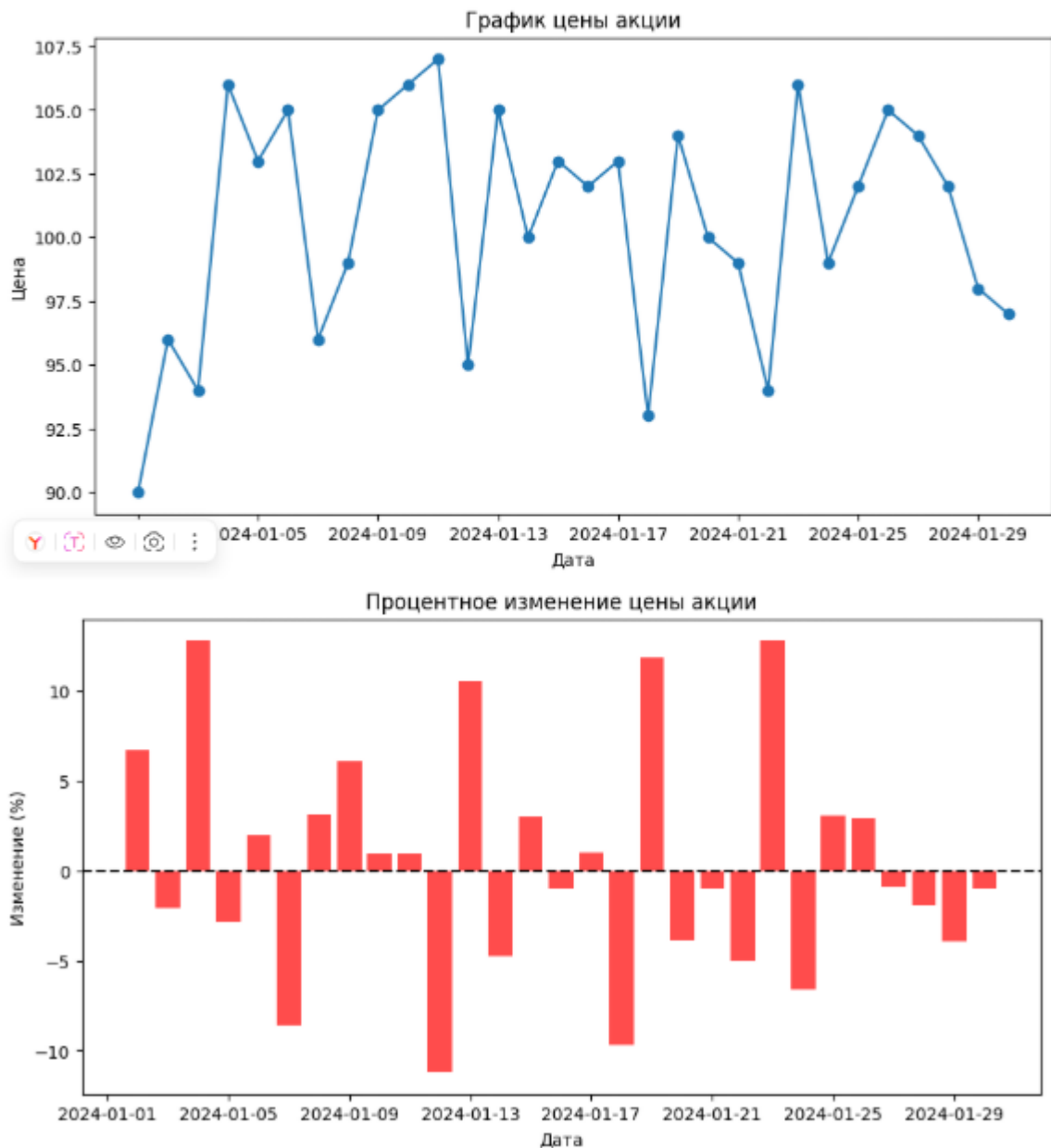


Рисунок 64. Пример 83

5. Были выполнены практические задания.

1) Создание Series из списка

Создайте Series из списка чисел [5, 15, 25, 35, 45] с индексами ['a', 'b', 'c', 'd', 'e']. Выведите его на экран и определите его тип данных.

```
[3]: import pandas as pd

s = pd.Series([5, 15, 25, 35, 45], index=['a', 'b', 'c', 'd', 'e'])

print(s)
print(f"Тип данных: {s.dtype}")
```

a 5
b 15
c 25
d 35
e 45
dtype: int64
Тип данных: int64

Рисунок 65. Практическое задание 1

2) Получение элемента Series

Дан Series с индексами ['A', 'B', 'C', 'D', 'E'] и значениями [12, 24, 36, 48, 60]. Используйте .loc[] для получения элемента с индексом 'C' и .iloc[] для получения третьего элемента.

```
: import pandas as pd

s = pd.Series([12, 24, 36, 48, 60], index=['A', 'B', 'C', 'D', 'E'])

print("Исходный Series:")
print(s)

element_loc = s.loc['C']
print(f"1. Элемент с индексом 'C': {element_loc}")

element_iloc = s.iloc[2]
print(f"2. Третий элемент: {element_iloc}")
```

Исходный Series:
A 12
B 24
C 36
D 48
E 60
dtype: int64
1. Элемент с индексом 'C': 36
2. Третий элемент: 36

Рисунок 66. Практическое задание 2

3) Фильтрация данных с помощью логической индексации

Создайте Series из массива NumPy `np.array([4, 9, 16, 25, 36, 49, 64])`. Выберите только те элементы, которые больше 20, и выведите результат.

```
import pandas as pd
import numpy as np

arr = np.array([4, 9, 16, 25, 36, 49, 64])
s = pd.Series(arr)

print("Исходный Series:")
print(s)

filtered_s = s[s > 20]

print("Элементы больше 20:")
print(filtered_s)
```

```
Исходный Series:
0    4
1    9
2   16
3   25
4   36
5   49
6   64
dtype: int64
Элементы больше 20:
3    25
4    36
5    49
6    64
dtype: int64
```

Рисунок 67. Практическое задание 3

4) Просмотр первых и последних элементов

Создайте Series, содержащий 50 случайных целых чисел от 1 до 100 (используйте `np.random.randint`). Выведите первые 7 и последние 5 элементов с помощью `.head()` и `.tail()`.

```
import pandas as pd
import numpy as np

random_numbers = np.random.randint(1, 101, 50)
s = pd.Series(random_numbers)

print("Весь Series (50 элементов):")
print(s)

print("Первые 7 элементов:")
print(s.head(7))

print("Последние 5 элементов:")
print(s.tail(5))
```

Весь Series (50 элементов):

0	27
1	9
2	79
3	15
4	90
5	42
6	77
7	51
8	63
9	96
10	52
11	96
12	4
13	94
14	23
15	15
16	43
17	29
18	36
19	13
20	32
21	71
22	59
23	86
24	28
25	66

Рисунок 68. Практическое задание 4 (1 часть)

```

26    42
27    45
28    62
29    57
30     6
31    28
32    28
33    44
34    84
35    30
36    62
37    75
38    92
39    89
40    62
41    97
42     1
43    27
44    62
45    77
46     3
47    70
48    72
49    27
dtype: int32
Первые 7 элементов:
0     27
1      9
2     79
3     15
4     90
5     42
6     77
dtype: int32
Последние 5 элементов:
45     77
46      3
47     70
48     72
49     27
dtype: int32

```

Рисунок 69. Практическое задание 4 (2 часть)

5) Определение типа данных Series

Создайте Series из списка ['cat', 'dog', 'rabbit', 'parrot', 'fish']. Определите тип данных с помощью `.dtype`, затем преобразуйте его в category с помощью `.astype()`.

```

import pandas as pd

animals = ['cat', 'dog', 'rabbit', 'parrot', 'fish']
s = pd.Series(animals)

print("Исходный Series:")
print(s)

print(f"Тип данных до преобразования: {s.dtype}")

s_category = s.astype('category')

print("Series после преобразования в category:")
print(s_category)

```

Исходный Series:

```

0      cat
1      dog
2  rabbit
3  parrot
4      fish
dtype: object

```

Тип данных до преобразования: object

Series после преобразования в category:

```

0      cat
1      dog
2  rabbit
3  parrot
4      fish
dtype: category
Categories (5, object): ['cat', 'dog', 'fish', 'parrot', 'rabbit']

```

Рисунок 70. Практическое задание 5

6) Проверка пропущенных значений

Создайте Series с данными [1.2, np.nan, 3.4, np.nan, 5.6, 6.8]. Напишите код, который проверяет, есть ли в Series пропущенные значения (NaN), и выведите индексы таких элементов.

```
import pandas as pd
import numpy as np

data = [1.2, np.nan, 3.4, np.nan, 5.6, 6.8]
s = pd.Series(data)

print("Исходный Series:")
print(s)

has_nan = s.isna().any()
print(f"Есть ли пропущенные значения? {has_nan}")

nans = s[s.isna()].index
print(f"Индексы пропущенных значений: {nans}")
```

Исходный Series:

0	1.2
1	NaN
2	3.4
3	NaN
4	5.6
5	6.8

dtype: float64
Есть ли пропущенные значения? True
Индексы пропущенных значений: Index([1, 3], dtype='int64')

Рисунок 71. Практическое задание 6

7) Заполнение пропущенных значений

Используйте Series из предыдущего задания и замените все NaN на среднее значение всех непустых элементов. Выведите результат.

```

import pandas as pd
import numpy as np

data = [1.2, np.nan, 3.4, np.nan, 5.6, 6.8]
s = pd.Series(data)

print("Исходный Series:")
print(s)

mean_val = s.mean()
print(f"Среднее значение непустых элементов: {mean_val}")

s_filled = s.fillna(mean_val)

print("Series после заполнения NaN средним значением:")
print(s_filled)

```

```

Исходный Series:
0    1.2
1    NaN
2    3.4
3    NaN
4    5.6
5    6.8
dtype: float64
Среднее значение непустых элементов: 4.25
Series после заполнения NaN средним значением:
0    1.20
1    4.25
2    3.40
3    4.25
4    5.60
5    6.80
dtype: float64

```

Рисунок 72. Практическое задание 7

8) Арифметические операции с Series

Создайте два Series:

```
s1 = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])
```

```
s2 = pd.Series([5, 15, 25, 35], index=['b', 'c', 'd', 'e'])
```

Выполните сложение $s1 + s2$. Объясните, почему в результате появляются NaN, и замените их на 0.

```
import pandas as pd

s1 = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])
s2 = pd.Series([5, 15, 25, 35], index=['b', 'c', 'd', 'e'])

print("s1:")
print(s1)

print("s2:")
print(s2)

s_sum = s1 + s2
print("Результат сложения s1 + s2:")
print(s_sum)

print("Объяснение появления NaN:")
print("Общие индексы: ['b', 'c', 'd']")
print("Индексы только в s1: ['a'] -> NaN")
print("Индексы только в s2: ['e'] -> NaN")

s_sum_filled = s_sum.fillna(0)
print("Результат после замены NaN на 0:")
print(s_sum_filled)
```

Рисунок 73. Практическое задание 8 (1 часть)

```

s1:
a    10
b    20
c    30
d    40
dtype: int64
s2:
b     5
c    15
d    25
e    35
dtype: int64
Результат сложения s1 + s2:
a     NaN
b    25.0
c    45.0
d    65.0
e     NaN
dtype: float64
Объяснение появления NaN:
Общие индексы: ['b', 'c', 'd']
Индексы только в s1: ['a'] -> NaN
Индексы только в s2: ['e'] -> NaN
Результат после замены NaN на 0:
a     0.0
b    25.0
c    45.0
d    65.0
e     0.0
dtype: float64

```

Рисунок 74. Практическое задание 8 (2 часть)

9) Применение функции к Series

Создайте Series из чисел [2, 4, 6, 8, 10]. Напишите код, который применяет к каждому элементу функцию вычисления квадратного корня с помощью `apply(np.sqrt)`.

```

import pandas as pd
import numpy as np

s = pd.Series([2, 4, 6, 8, 10])

print("Исходный Series:")
print(s)

s_sqrt = s.apply(np.sqrt)

print("Квадратный корень каждого элемента:")
print(s_sqrt)

```

```

Исходный Series:
0    2
1    4
2    6
3    8
4   10
dtype: int64
Квадратный корень каждого элемента:
0    1.414214
1    2.000000
2    2.449490
3    2.828427
4    3.162278
dtype: float64

```

Рисунок 75. Практическое задание 9

10) Основные статистические методы

Создайте Series из 20 случайных чисел от 50 до 150 (используйте `np.random.randint`). Найдите сумму, среднее, минимальное и максимальное значение. Выведите также стандартное отклонение.

```

import pandas as pd
import numpy as np

random_numbers = np.random.randint(50, 151, 20)
s = pd.Series(random_numbers)

print("Созданный Series:")
print(s)

print("Основные статистические характеристики:")
print(f"Сумма: {s.sum()}")
print(f"Среднее: {s.mean():.2f}")
print(f"Минимальное значение: {s.min()}")
print(f"Максимальное значение: {s.max()}")
print(f"Стандартное отклонение: {s.std():.2f}")

```

```

Созданный Series:
0    147
1    142
2     71
3     91
4     62
5    129
6    111
7    128
8    101
9    132
10    78
11    143
12    54
13    68
14    150
15    77
16    95
17    86
18    134
19    104
dtype: int32
Основные статистические характеристики:
Сумма: 2103
Среднее: 105.15
Минимальное значение: 54
Максимальное значение: 150
Стандартное отклонение: 31.18

```

Рисунок 76. Практическое задание 10

11) Работа с временными рядами

Создайте Series, где индексами будут даты с 1 по 10 марта 2024 года (`pd.date_range(start='2024-03-01', periods=10, freq='D')`). Значения — случайные числа от 10 до 100. Выберите данные за 5–8 марта.

```

import pandas as pd
import numpy as np

date_range = pd.date_range(start='2024-03-01', periods=10, freq='D')

random_values = np.random.randint(10, 101, 10)
s = pd.Series(random_values, index=date_range)

print("Временной ряд (1-10 марта 2024):")
print(s)

print("Данные за 5-8 марта:")
print(s['2024-03-05':'2024-03-08'])

```

Временной ряд (1-10 марта 2024):

2024-03-01	74
2024-03-02	36
2024-03-03	76
2024-03-04	40
2024-03-05	22
2024-03-06	79
2024-03-07	71
2024-03-08	92
2024-03-09	64
2024-03-10	64

Freq: D, dtype: int32

Данные за 5-8 марта:

2024-03-05	22
2024-03-06	79
2024-03-07	71
2024-03-08	92

Freq: D, dtype: int32

Рисунок 77. Практическое задание 11

12) Проверка уникальности индексов

Создайте Series с индексами ['A', 'B', 'A', 'C', 'D', 'B'] и значениями [10, 20, 30, 40, 50, 60]. Проверьте, являются ли индексы уникальными. Если нет, сгруппируйте повторяющиеся индексы и сложите их значения.

```
import pandas as pd

s = pd.Series([10, 20, 30, 40, 50, 60], index=['A', 'B', 'A', 'C', 'D', 'B'])

print("Исходный Series с повторяющимися индексами:")
print(s)

is_unique = s.index.is_unique
print(f"Индексы уникальны? {is_unique}")

if not is_unique:
    print("Индексы не уникальны. Выполняем группировку и суммирование:")
    s_grouped = s.groupby(s.index).sum()
    print(s_grouped)
else:
    print("Индексы уникальны. Группировка не требуется.")
```

```
Исходный Series с повторяющимися индексами:
A    10
B    20
A    30
C    40
D    50
B    60
dtype: int64
Индексы уникальны? False
Индексы не уникальны. Выполняем группировку и суммирование:
A    40
B    80
C    40
D    50
dtype: int64
```

Рисунок 78. Практическое задание 12

13) Преобразование строковых дат в DateTimeIndex

Создайте Series, где индексами будут строки ['2024-03-10', '2024-03-11', '2024-03-12']. Значения [100, 200, 300]. Преобразуйте индексы в DateTimeIndex и выведите тип данных индекса.

```
import pandas as pd

s = pd.Series([100, 200, 300], index=['2024-03-10', '2024-03-11', '2024-03-12'])

print("Исходный Series со строковыми индексами:")
print(s)

s.index = pd.to_datetime(s.index)

print("Series после преобразования индексов в DatetimeIndex:")
print(s)

print(f"Тип данных индекса после преобразования: {type(s.index)}")
```

Исходный Series со строковыми индексами:

2024-03-10	100
2024-03-11	200
2024-03-12	300

dtype: int64

Series после преобразования индексов в DatetimeIndex:

2024-03-10	100
2024-03-11	200
2024-03-12	300

dtype: int64

Тип данных индекса после преобразования: <class 'pandas.core.indexes.datetimes.DatetimeIndex'>

Рисунок 79. Практическое задание 13

14) Чтение данных из CSV-файла

Создайте CSV-файл data.csv со следующими данными:

Дата, Цена

2024-03-01, 100

2024-03-02, 110

2024-03-03, 105

2024-03-04, 120

2024-03-05, 115

Прочитайте файл и создайте Series, используя "Дата" в качестве индекса.

```
import pandas as pd

data = """Дата,Цена
2024-03-01,100
2024-03-02,110
2024-03-03,105
2024-03-04,120
2024-03-05,115
"""

with open('data.csv', 'w', encoding='utf-8') as f:
    f.write(data)

s = pd.read_csv("data.csv", index_col="Дата")["Цена"]

print(s)
```

Дата	Цена
2024-03-01	100
2024-03-02	110
2024-03-03	105
2024-03-04	120
2024-03-05	115

Name: Цена, dtype: int64

Рисунок 80. Практическое задание 14

15) Построение графика на основе Series

Создайте Series, где индексами будут даты с 1 по 30 марта 2024 года, а значениями – случайные числа от 50 до 150. Постройте график значений с помощью matplotlib. Добавьте заголовок, подписи осей и сетку.

```

: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

s = pd.Series(np.random.randint(50, 151, 30), index=pd.date_range('2024-03-01', periods=30, freq='D'))

plt.plot(s.index, s.values, marker='o')
plt.title('График значений')
plt.xlabel('Дата')
plt.ylabel('Значение')
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```



Рисунок 81. Практическое задание 15

6. Было выполнено индивидуальное задание.

Продажи в интернет-магазине: загрузка, анализ, визуализация

Создайте CSV-файл sales.csv со следующими данными:

Дата, Продажи

2024-07-01, 120

2024-07-02, 150

2024-07-03, 170

2024-07-04, 160

2024-07-05, 180

Прочитайте данные, установите DatetimeIndex, найдите скользящее среднее (rolling(2).mean()) и постройте два графика: реальных продаж и их сглаженного значения.

```
: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

data = """Дата,Продажи
2024-07-01,120
2024-07-02,150
2024-07-03,170
2024-07-04,160
2024-07-05,180
"""

with open('sales.csv', 'w', encoding='utf-8') as f:
    f.write(data)

df = pd.read_csv('sales.csv', index_col='Дата', parse_dates=['Дата'])
s = df['Продажи']

print("Исходные данные:")
print(s)

s_rolling = s.rolling(2).mean()

print("Скользящее среднее:")
print(s_rolling)

plt.figure(figsize=(12, 6))

plt.plot(s.index, s.values, marker='o', label='Реальные продажи')

plt.plot(s_rolling.index, s_rolling.values, marker='s', color='red', linestyle='--', label='Скользящее среднее')

plt.title('Продажи в интернет-магазине', fontsize=16)

plt.xlabel('Дата', fontsize=12)
plt.ylabel('Продажи', fontsize=12)

plt.grid(True, alpha=0.3)

plt.legend(loc='upper left', fontsize=12)

plt.tight_layout()

plt.show()

Исходные данные:
Дата
2024-07-01    120
2024-07-02    150
2024-07-03    170
2024-07-04    160
2024-07-05    180
Name: Продажи, dtype: int64
Скользящее среднее:
Дата
2024-07-01     NaN
2024-07-02    135.0
2024-07-03    160.0
2024-07-04    165.0
2024-07-05    170.0
Name: Продажи, dtype: float64
```

Рисунок 82. Индивидуальное задание (1 часть)

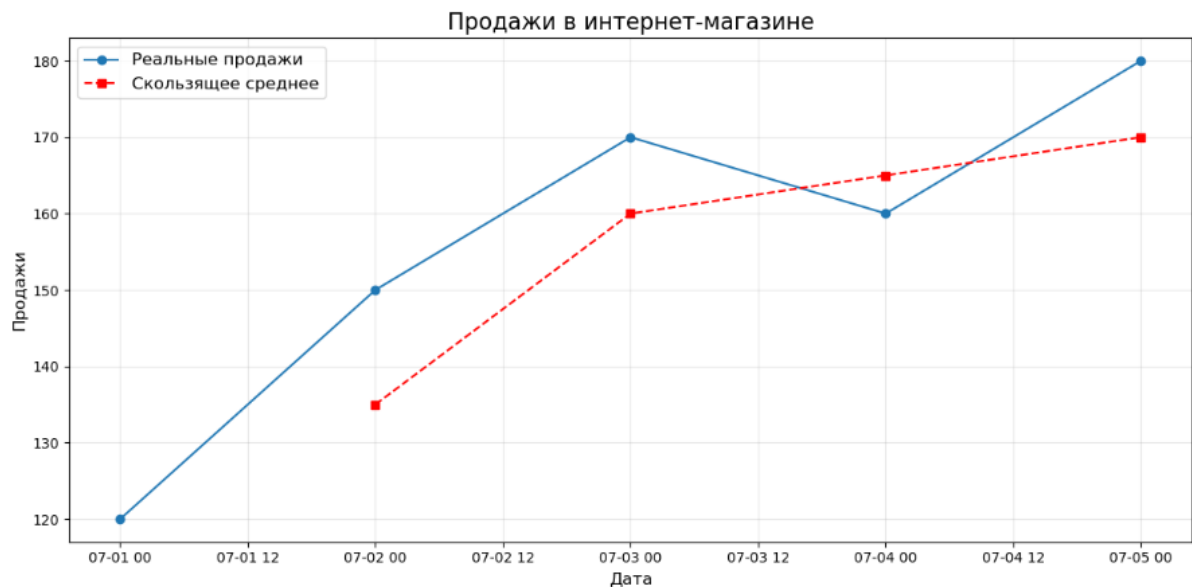


Рисунок 83. Индивидуальное задание (2 часть)

Контрольные вопросы:

1. Series – это одномерный массив данных с метками, которые называются индексами. Отличие от списка в том, что у каждого элемента Series есть своя метка (индекс), а в списке элементы доступны только по числовой позиции.
2. Можно использовать числа целые и дробные, строки, логические значения True и False, даты и время, а также массивы NumPy.
3. При создании Series нужно передать параметр index со списком меток. Например: `pd.Series([10, 20, 30], index=['a', 'b', 'c'])`.
4. Нужно написать имя Series и в квадратных скобках указать метку индекса. Например: `s['a']`.
5. `.loc[]` работает с метками индексов, а `.iloc[]` работает с числовыми позициями. `.loc['a']` вернёт элемент с меткой 'a', а `.iloc[0]` вернёт первый элемент независимо от его метки.
6. Нужно внутри квадратных скобок написать условие. Например: `s[s > 10]` вернёт только те элементы, которые больше 10.
7. Метод `.head()` показывает первые несколько элементов, а `.tail()` показывает последние несколько элементов. По умолчанию они показывают 5 элементов, но можно указать любое число.

8. Нужно использовать свойство `.dtype`. Например: `s.dtype` покажет тип данных всех элементов Series.

9. Нужно использовать метод `.astype()`. Например: `s.astype('float')` превратит целые числа в дробные.

10. Можно использовать метод `.isna()` или `.isnull()`. Они возвращают True там, где есть пропуски. А метод `.hasnans` показывает, есть ли хотя бы один пропуск.

11. Основной метод – `.fillna()`. Он заменяет пропуски на указанное значение. Например, `s.fillna(0)` заменит все пропуски на ноль.

12. `.fillna()` заменяет пропущенные значения на какое-то другое значение, сохраняя все элементы. `.dropna()` полностью удаляет строки с пропущенными значениями.

13. Можно складывать, вычитать, умножать, делить Series друг на друга или на обычные числа. Также доступны возведение в степень, вычисление квадратного корня и другие операции.

14. Векторизованные операции работают сразу со всеми элементами одновременно, что делает код короче и значительно быстрее, особенно на больших данных. Циклы обрабатывают элементы по одному и работают медленнее.

15. Нужно использовать метод `.apply()`. Например: `s.apply(lambda x: x * 2)` умножит каждый элемент на 2.

16. Доступны функции: `.sum()` для суммы, `.mean()` для среднего, `.min()` для минимума, `.max()` для максимума, `.count()` для количества элементов, `.std()` для стандартного отклонения.

17. Используйте методы: `s.min()`, `s.max()`, `s.mean()`, `s.std()`.

18. `.sort_values()` сортирует по значениям, а `.sort_index()` сортирует по индексам.

19. Нужно использовать свойство `.index.is_unique`. Оно вернёт True, если все индексы разные, и False, если есть повторяющиеся.

20. Нужно использовать метод `.reset_index()`. Он превращает текущие индексы в обычный столбец и создаёт новые числовые индексы от 0.

21. Нужно присвоить новый список меток свойству `.index`. Например: `s.index = ['x', 'y', 'z']`.

22. Временные ряды создаются с помощью `pd.date_range()` для создания дат. Затем эти даты передаются как индекс Series. После этого можно выбирать данные по датам, использовать срезы по датам и применять специальные методы для работы с временными данными.

23. Нужно использовать `pd.to_datetime()`. Например: `s.index = pd.to_datetime(s.index)`. Это превращает строки с датами в специальный тип данных для работы с датами.

24. Можно использовать срез по датам: `s['2024-03-01':'2024-03-10']` выберет все данные с 1 по 10 марта.

25. Сначала читаем весь файл через `pd.read_csv()` в DataFrame, затем выбираем нужный столбец. Например: `df = pd.read_csv('file.csv'); s = df['название_столбца']`.

26. При чтении файла можно использовать параметр `index_col`: `df = pd.read_csv('file.csv', index_col='название_столбца')`. Затем извлечь нужный столбец в Series.

27. Этот метод вычисляет скользящее среднее. Он берёт окно из нескольких элементов и считает среднее по этому окну, затем окно сдвигается. Это помогает сгладить колебания данных и увидеть общий тренд.

28. `.pct_change()` вычисляет процентное изменение между текущим и предыдущим элементом. Он показывает, на сколько процентов выросло или упало значение по сравнению с прошлым периодом. Это полезно для расчёта доходности или темпов роста.

29. `.rolling()` полезен для сглаживания шумов на графиках и поиска трендов в данных. `.pct_change()` полезен для расчёта доходности акций, темпов роста продаж, инфляции и любых других данных, где важно относительное изменение.

30. NaN появляются, когда при сложении двух Series индексы не совпадают, при чтении данных с пустыми ячейками в файле, или при делении на ноль. Работать с ними можно так: `.isna()` для поиска пропусков, `.fillna()` для замены пропусков на нужное значение, `.dropna()` для удаления строк с пропусками.

Вывод: были изучены основы работы с библиотекой pandas, в частности, со структурой данных Series.